MODUL 2. REVIEW ALGORITMA DAN PEMROGRAMAN 1

2.1 Struktur Program Go

Dalam kerangka program yang ditulis dalam bahasa pemrograman Go, program utama selalu mempunyai dua komponen berikut:

- **package main** merupakan penanda bahwa file ini berisi program utama.
- **func main()** berisi kode utama dari sebuah program Go.

Komentar, bukan bagian dari kode program, dan dapat ditulis di mana saja di dalam program:

- Satu baris teks yang diawali dengan garis miring ganda ('//') s.d. akhir baris, atau.
- Beberapa baris teks yang dimulai dengan pasangan karakter '/*' dan diakhiri dengan '*/'.

```
// Setiap program utama dimulai dengan "package main"
package main

// Impor paket yang dibutuhkan, "fmt" berisi proses I/O standar
import “fmt”

// Kode program utama dalam "fungsi main"
func main() {
    …
}
```

School of Computing
Telkom University
Informatics lab

Contoh sebuah program dalam bahasa pemrograman Go (nama file hello.go).

```
package main
import "fmt"
func main() {
    var greetings = "Selamat datang di dunia DAP"
    var a, b int

    fmt.Println(greetings)
    fmt.Scanln(&a, &b)
    fmt.Printf("%v + %v = %v\n", a, b, a+b)
}
```

```
C:\usvus\go\suc\hvllo>aiu
Diuvctouy of C:\usvus\jiuuyt\go\suc\hvllo
 6/29/2019   7:15 flM          1,727   hvllo.go
C:\usvus\go\suc\hvllo>go buila hvllo.go
C:\usvus\go\suc\hvllo>aiu
Diuvctouy of C:\usvus\jiuuyt\go\suc\hvllo
 6/29/2019   7:15 flM          1,727   hvllo.go
 6/29/2019   7:18 flM      2,198,528   hvllo.vxv
C:\usvus\go\suc\hvllo>hvllo
Svlauat aatatg ai autia DAfl
7 5
7 + 5 = 12
C:\usvus\go\suc\hvllo>
```

1) **Koding, Kompilasi, dan Eksekusi Go**

Koding

- Tidak berbeda dengan penulisan program sumber dalam bahasa lain, program Go harus dibuat menggunakan penyunting teks dan disimpan dalam format teks, bukan dalam format dokumen (doc, docx, atau lainnya).
- Setiap program go disimpan dalam file teks dengan ekstensi *.go, dengan nama bebas. Sebaiknya nama file adalah nama untuk program tersebut.
- Setiap satu program lengkap Go disimpan dalam satu folder tersendiri. Nama folder merupakan nama program tersebut. Karena itu secara prinsip, satu program Go dapat dipecah dalam beberapa file dengan ekstensi *.go selama disimpan dalam folder yang sama.

Kompilasi

Beberapa bahasa pemrograman dirancang untuk diimplementasikan sebagai interpreter dan lainnya sebagai kompilator. Interpreter akan membaca setiap baris instruksi dan kemudian langsung mengeksekusinya, dengan hanya sedikit pemeriksaan apakah penulisan keseluruhan program sudah benar atau belum. Kompilator akan memeriksa keseluruhan program sumber dan kemudian mengubahnya menjadi program eksekutabel, sehingga konsistensi penulisan (seperti penggunaan tipe data) sudah diperiksa sebelum eksekusi. Selain itu karena program dibuat menjadi eksekutabel lebih dahulu, proses optimasi dapat dilakukan sehingga program menjadi sangat efisien.

Fakultas Informatika
School of Computing
Telkom University

Go diimplementasikan sebagai kompilator. Berikut adalah contoh sesi yang biasa dilakukan saat mengkompilasi dan mengeksekusi program dalam bahasa Go:

- Panggil shell atau terminal (program/utiliti cmd.exe di Windows)
- Masuk ke dalam (cd) folder program (normalnya ada di C:\Users\go\src\ atau yang sejenis)
- Kemudian panggil perintah go build atau go build file.go untuk mengkompilasi file.go
- Jika gagal, akan muncul pesan eror yang sesuai, pelajari dengan baik pesan tersebut, perbaiki teks program sumber, kemudian ulangi proses build-nya.
- Jika berhasil maka pada folder tersebut akan dibuat program dengan nama yang sama dan diakhiri dengan .exe (untuk Windows)

- Panggil program eksekutabel tersebut dari terminal yang sama. Jangan memanggil program tersebut dengan mengklik eksekutabel tersebut dari folder karena program kalian hanya berbasis teks, bukan/belum dirancang dengan tampilan Windows.

Catatan

Semua proses terkait bahasa Go dilakukan melalui utilitas go. Beberapa opsi dengan utilitas go:

- **go build**: mengkompilasi program sumber yang ada dalam folder menjadi sebuah program.
- **go build file.go**: mengkompilasi program sumber file.go saja.
- **go fmt**: membaca semua program sumber dalam folder dan mereformat penulisannya agar sesuai dengan standar penulisan program sumber Go.
- **go clean**: membersihkan file-file dalam folder sehingga tersisa program sumber nya saja.

Video tutorial memulai program Go: https://youtu.be/S5Rt2qO2QEg

2) **Latihan**

Fakultas Informatika
School of Computing
Telkom University
informatics lab

1. Selidiki bahasa-bahasa pemrograman berikut, apakah termasuk diinterpretasi, dikompilasi, dikompilasi (ke instruksi perantara) kemudian diinterpretasi:
    - Pascal
    - C dan C++
    - Java
    - Python
2. Unduh kompilator Go di komputer yang Anda gunakan. kemudian salin contoh program di atas ke dalam folder C:\Users\userid\Go\hello\hello.go, yaitu buat folder Go dalam direktori home Anda, kemudian buat subfolder hello dan taruh file hello.go di dalamnya. Hidupkan terminal (cmd.exe), dan panggil go build di dalam folder hello tersebut. Periksa apakah hello.exe muncul di folder tersebut? Jika ya, coba eksekusi program tersebut, juga melalui terminal cmd.exe tersebut. (Jangan di klik melalui browser folder).

2.2 Tipe Data dan Instruksi Dasar

1) Data dan Variabel

Variabel adalah nama dari suatu lokasi di memori, yang data dengan tipe tertentu dapat disimpan.

- Nama variabel dimulai dengan huruf dan dapat dikuti dengan sejumlah huruf, angka, atau garisbawah.

  Contoh: **ketemu**, **found**, **rerata**, **mhs1**, **data_2**, **…**

Notasi tipe dasar	Tipe dalam Go	Keterangan
integer	int int8 int32 //rune int64 uint uint8 //byte uint32 uint64	bergantung platform 8 bit: -128..127 32 bit: -10^9..10^9 64 bit: -10^19..10^19 bergantung platform 0..255 0..4294967295 0..(2^64-1)
real	float32 float64	32bit: -3.4E+38 .. 3.4E+38 64bit: -1.7E+308 .. 1.7E+308
boolean (atau logikal)	**bool**	**false** dan **true**
karakter	byte //uint8 rune //int32	tabel ASCII/UTF-8 tabel UTF-16
string	string	

Fakultas Informatika
School of Computing
Telkom University
informatics lab

- Tipe data yang umum tersedia adalah integer, real, boolean, karakter, dan string. Lihat tabel berikut ini untuk variasi tipe data yang disediakan dalam bahasa Go.
- **Nilai data** yang tersimpan dalam variabel dapat diperoleh dengan menyebutkan langsung nama variabelnya.

  Contoh: Menyebutkan nama **found** akan mengambil nilai tersimpan dalam memori untuk variabel **found**, pastinya.
- **Informasi alamat** atau lokasi dari variabel dapat diperoleh dengan menambahkan prefiks **&** di depan nama variabel tersebut.

  Contoh: **&found** akan mendapatkan alamat memori untuk menyimpan data pada **found**.
- Jika variabel berisi alamat memori, prefiks ***** pada variabel tersebut akan memberikan nilai yang tersimpan dalam memori yang lokasinya disimpan dalam variabel tersebut.

Contoh: ***mem** akan mendapatkan data di memori yang alamatnya tersimpan di **mem.** Karenanya ***(&found)** akan mendapatkan data dari lokasi memori variabel **found** berada, alias sama saja dengan menyebutkan langsung **found** 8=).

- Operasi yang dapat dilakukan terhadap tipe data di atas adalah

Operator dalam Go	Tipe data terkait	Keterangan
+	string integer dan real	konkatenasi 2 string operasi penjumlahan
- * /	integer dan real	operasi pengurangan, perkalian, dan pembagian
%	integer	operasi sisa pembagian integer (modulo)
& | ^ &^	integer	operasi **per-bit** AND, OR, XOR, AND-NOT
<< >>	integer dan unsigned integer	operasi geser bit kiri/kanan sebanyak unsigned integer yang diberikan
< <= >= > == !=	selain boolean	komparasi menghasilkan nilai boolean komparasi karakter sesuai dengan posisi karakter tersebut dalam tabel ASCII/UTF-16 komparasi string sesuai dengan operasi karakter per karakter, dimulai dari karakter paling kiri (awal)
&& || !	boolean	operasi **boolean** AND, OR, dan NOT
* &	variabel apasaja	mendapatkan data dari lokasi memori dan mendapatkan lokasi dari variabel

Fakultas Informatika
School of Computing
Telkom University
informatics.lab

Contoh:

Operasi	Hasil
"non suffi" + "cit mundo"	"non sufficit mundo"
2019.01 + 1.0102	2020.0202
2020 / 20	22.22
20.2 * 1.1	101
2020 % 1999	21
2020 & 1111	2104
2020 ^ 1111	1663
2020 >> 2	505
"minutus" < "magnus"	false
2020 >= 1234	true
! false && true	true

- Bahasa Go menganut kesesuaian tipe data yang ketat. Tipe data yang berbeda tidak boleh dicampur dalam satu ekspresi, bahkan tipe data masih yang sejenis, misalnya masih sama-sama integer (**int** dan **int32**). Untuk menyesuaikan tipe data, ada beberapa cara yang dapat dilakukan:
  - ✓ Casting, **tipe(data),** mengubah tipe dari data yang diberikan ke tipe yang diinginkan.
  - ✓ Memanfaatkan fungsi **Sprint** dan **Sscan** dari paket **fmt**.
  - ✓ Memanfaatkan fungsi-fungsi dalam paket **strconv**, seperti **Atoi**, **Itoa**, dan **ParseBool**. Lihat lampiran untuk contoh penggunaan.

  Contoh:

Operasi	Hasil
2020.0 % 19	will be an illegal expression error
int(2020.0) % 19	6

Konversi tipe	Data	Tipe baru	Keterangan
tipe(data)	integer	integer	format data tidak berubah, hanya penyesuaian jumlah bit. Kekurangan bit diisi bit 0 di sebelah kiri (MSB)
	real	real	format data tidak berubah, hanya penyesuaian jumlah bit. Kekurangan bit, maka bit mantisa diisi bit 0.
	real	integer	format data disesuaikan dengan tipe data tujuan
	integer	real	format data disesuaikan dengan tipe data tujuan
fmt.Sprintf("%v",v)	any type	string	tulis output ke string
fmt.Sprintf("%c",v)	karakter	string	tulis karakter ke string
fmt.Sscanf(s,"%v",&v)	string	any type	baca string ke variabel dengan tipe tertentu
fmt.Sscanf(s,"%c",&v)	string	karakter	baca string ke variabel bertipe karakter

Fakultas Informatika
School of Computing
Telkom University
Informatics lab

- Variabel harus dideklarasikan dulu sebelum digunakan. Variabel juga harus diinisialisasi dulu (diisi data) agar nilai yang tersimpan diketahui dengan jelas dan eksekusi algoritma menjadi terprediksi. Dalam bahasa Go, variabel yang tidak diinisialisasi lebih dahulu otomatis diisi dengan nilai default yang ekuivalen dengan bit 0.
  - ✓ Nilai 0 untuk bilangan integer
  - ✓ 0.0E+0 untuk bilangan real

- ✓ **false** untuk boolean
- ✓ Karakter NUL (lihat tabel ASCII) untuk karakter
- ✓ "" (string kosong, string dengan panjang 0) untuk string
- ✓ **nil** untuk alamat memori

Notasi deklarasi variabel	Penulisan dalam Go	Keterangan
kamus a : tipe	var a tipe	a diinisialisasi dengan nilai default
kamus a : tipe algoritma a <- nilai_awal	var a tipe = nilai_awal var a = (tipe)nilai_awal	a diinisialisasi dengan nilai_awal
	a := nilai_awal a := (tipe)nilai_awal	secara **implisit**, tipe variabel a ditentukan dari nilai inisialisasinya

Contoh:

```
1  func main() {
2      var a int
3      a = 2019
4      r := 2019.0707
5      b := false
6      c := 'x'
7      s := "a string is a string"
8  }
```

2) **Instruksi Dasar**

Notasi instruksi dasar	Penulisan dalam bahasa Go	Keterangan
v1 <- e1 v1 <- v1 + e1 v1 <- v1 - e1 v1 <- v1 + 1 v1 <- v1 - 1	v1 = e1 v1 += e1 // atau v1 = v1 + e1 v1 -= e1 // atau v1 = v1 - e1 v1++ // atau v1 = v1 + 1 v1-- // atau v1 = v1 - 1	operasi assignment, mengisi data ke lokasi memori (variabel)
input(v1, v2)	fmt.Scan(&v1, &v2) fmt.Scanln(&v1, &v2) fmt.Scanf("%v %v", &v1, &v2)	Pembacaan data memerlukan alamat memori ke mana data akan disimpan.
output(e1, e2)	fmt.Print(e1, e2) fmt.Println(e1, e2) fmt.Printf("%v %v\n", e1, e2)	Penulisan data memerlukan nilai data yang akan ditulis.

Contoh:

```
package main
import "fmt"

func main() {
    var a, b, c float64
    var hipotenusa bool

    fmt.Scanln( &a, &b, &c )
    hipotenusa = (c*c) == (a*a + b*b)
    fmt.Println( "Sisi c adalah hipotenusa segitiga a,b,c: ", hipotenusa)
}
```

3) **Konstanta Simbolik**

Konstanta dapat diberi nama untuk memudahkan mengingat maksud dan manfaat dari nilai yang diberi nama tersebut. Seperti PI untuk merepresentasikan konstanta π.

```
const PI = 3.14159265358979323846264 33
const MARKER = "AKHIR"
```

4) **Soal Latihan Modul 2A**

Fakultas Informatika
School of Computing
Telkom University
informatics lab

1. Telusuri program berikut dengan cara mengkompilasi dan mengeksekusi program. Silakan masukan data yang sesuai sebanyak yang diminta program. Perhatikan keluaran yang diperoleh. Coba terangkan apa sebenarnya yang dilakukan program tersebut?

```
package main
import "fmt"

func main() {
    var (
        satu, dua, tiga string
        temp string
    )
    fmt.Print("Masukan input string: ")
    fmt.Scanln(&satu)
    fmt.Print("Masukan input string: ")
    fmt.Scanln(&dua)
    fmt.Print("Masukan input string: ")
    fmt.Scanln(&tiga)
    fmt.Println("Output awal = " + satu + " " + dua + " " + tiga)
    temp = satu
    satu = dua
    dua = tiga
    tiga = temp
    fmt.Println("Output akhir = " + satu + " " + dua + " " + tiga)
}
```

2. Tahun kabisat adalah tahun yang habis dibagi 400 atau habis dibagi 4 tetapi tidak habis dibagi 100. Buatlah sebuah program yang menerima input sebuah bilangan bulat dan memeriksa apakah bilangan tersebut merupakan tahun kabisat (**true**) atau bukan (**false**).

   (Contoh input/output, **Teks bergaris bawah** adalah input dari user):

1	Hahut: **2016** Kabisat: tuuv
2	Hahut: **2000** Kabisat: tuuv
3	Hahut: **2018** Kabisat: falsv

3. Buat program **Bola** yang menerima input jari-jari suatu bola (bilangan bulat). Tampilkan Volume dan Luas kulit bola. $volumebola = \frac{4}{3}\pi r^3$ dan $luasbola = 4\pi r^2$ (π ≈ 3.1415926535).

   (Contoh input/output, **Teks bergaris bawah** adalah input dari user):

```
Jvjaui = 5
Bola avtgat jvjaui 5 uvuiliei voluuv 523.5988 aat luas eulit 314.1593
```

Fakultas Informatika
School of Computing
Telkom University
informatics lab

4. Dibaca nilai temperatur dalam derajat Celsius. Nyatakan temperatur tersebut dalam Fahrenheit

$$Celsius = (Fahrenheit - 32) \times \frac{5}{9} \quad Reamur = Celcius \times \frac{4}{5} \quad Kelvin = (Fahrenheit + 459.67) \times \frac{5}{9}$$

   (Contoh input/output, **Teks bergaris bawah** adalah input dari user):

```
Hvupvuatuu Cvlsius: 50
Dvuajat Fahuvthvit: 122
```

   Lanjutkan program di atas, sehingga temperatur dinyatakan juga dalam derajat Reamur dan Kelvin.

   (Contoh input/output, **Teks bergaris bawah** adalah input dari user):

```
Hvupvuatuu Cvlsius: 50
Dvuajat Rvauuu: 40
Dvuajat Fahuvthvit: 122
Dvuajat Kvlvit: 323
```

5. Tipe karakter sebenarnya hanya apa yang tampak dalam tampilan. Di dalamnya tersimpan dalam bentuk biner 8 bit (byte) atau 32 bit (rune) saja.

Buat program ASCII yang akan membaca 5 buat data integer dan mencetaknya dalam format karakter. Kemudian membaca 3 buah data karakter dan mencetak 3 buah karakter setelah karakter tersebut (menurut tabel ASCII)

Masukan terdiri dari dua baris. Baris pertama berisi 5 buah data integer. Data integer mempunyai nilai antara 32 s.d. 127. Baris kedua berisi 3 buah karakter yang berdampingan satu dengan yang lain (tanpa dipisahkan spasi).

Keluaran juga terdiri dari dua baris. Baris pertama berisi 5 buah representasi karakter dari data yang diberikan, yang berdampingan satu dengan lain, tanpa dipisahkan spasi. Baris kedua berisi 3 buah karakter (juga tidak dipisahkan oleh spasi).

No.	Masukat	Keluarat
1	66 97 103 117 115 SNO	Bagus HOfl

Catatan: Gunakan fmt.Scanf("%c", &var) untuk pembacaan satu karakter dan fmt.Printf("%c", var) untuk penulisan satu karakter.

Fakultas Informatika
School of Computing
Telkom University
informatics lab

2.3 Struktur Kontrol Perulangan

Go hanya mempunyai kata kunci **for** untuk semua jenis perulangan yang kita pelajari dalam notasi algoritma. Dua bentuk yang kita gunakan di sini adalah struktur **while-loop** dan **repeat-until**.

Bentuk perulangan dalam bahasa Go

```
for inisialisasi; kondisi; update {
    // .. for-loop ala C
    // .. ke-3 bagian opsional, tetapi ";" tetap harus ada
}
for kondisi {
    // .. ulangi kode di sini selama kondisi terpenuhi
    // .. sama seperti "for ; kondisi; {"
}
for {
    // .. tanpa kondisi, berarti loop tanpa henti (perlu if-break)
}
for ndx, var := range slice/array {
    // .. iterator mengunjungi seluruh isi slice/array
    // .. pada setiap iterasi ndx diset indeks dan var diisi nilainya
}
```

Dalam konsep pemrograman terstruktur, setiap rancangan algoritma harus memenuhi syarat satu pintu masuk dan satu pintu keluar. Karena itu **tidaklah diperkenankan** untuk membuat program sumber yang mempunyai struktur loop yang mempunyai pintu keluar lebih dari satu, seperti:

Fakultas Informatika
School of Computing
Telkom University
informatics lab

- Satu pintu keluar dari kondisi **for** dan satu lagi dari instruksi **if-break**
- Atau mempunyai instruksi **if-break** yang lebih dari satu.

1) **Bentuk While-Loop**

Bentuk **while-loop** memastikan setiap kali memasuki loop, ada kondisi yang harus terpenuhi (benar/**true**). Ini juga berarti saat keluar dari loop, maka nilai kondisi tersebut pasti salah/**false**!

	Notasi algoritma	Penulisan dalam bahasa Go
1	while (kondisi) do	for kondisi {
2		
3	... kode yang diulang	.. kode yang diulang
4	endwhile	}